Data Flow Through a MiniViT + DINOHead Network

April 30, 2026

Abstract

We trace a single image through the entire vision encoder architecture used in our self-supervised experiments, with concrete (made-up) numbers at every stage so the math is easy to follow. The architecture is UNI-like but is scaled down to fit our compute budget. The aim is to show *exactly* what arithmetic the data undergoes on the way through the network (forward pass) and what arithmetic the parameters undergo on the way back (backward pass and EMA teacher update).

1 Setup

Quantity	**Value**
Input image	$3 \times 96 \times 96$
Patch size	8×8
Number of patches	$12 \times 12 = 144$
Embedding dimension	$D = 192$
Attention heads	$h = 3$, with $d_h = 64$ per head
Transformer blocks	$L = 6$
DINO head output dim	$K = 128$

2 Forward Pass

2.1 Step 1 — Patch Embedding

The input is a single image with shape $1 \times 3 \times 96 \times 96$. A Conv2d($3 \to 192$, $k = 8$, $s = 8$) slides an 8×8 kernel with stride 8, producing

$$\frac{96}{8} \times \frac{96}{8} \;=\; 12 \times 12 \;=\; 144 \quad \text{patches.}$$

Each patch is mapped to a 192-dimensional vector. After flattening the spatial grid we have 144 tokens of dimension 192. Concretely, imagine the first few patch tokens look like

$$\mathbf{e}_1 = \begin{bmatrix} 0.23 \\ -1.05 \\ 0.87 \\ 0.44 \\ \vdots \\ -0.31 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 1.12 \\ 0.03 \\ -0.55 \\ 0.91 \\ \vdots \\ 0.67 \end{bmatrix}, \ \ldots, \ \mathbf{e}_{144} = \begin{bmatrix} -0.08 \\ 0.74 \\ 1.23 \\ -0.62 \\ \vdots \\ 0.15 \end{bmatrix},$$

each in $\mathbb{R}^{192}$.

2.2 Step 2 — CLS Token and Positional Embedding

A learnable CLS token $\mathbf{e}_{\text{cls}} \in \mathbb{R}^{192}$ is prepended, giving 145 tokens. We then add a learned positional vector to each token. For example, for patch 1:

$$\underbrace{\begin{bmatrix} 0.23 \\ -1.05 \\ 0.87 \\ \vdots \end{bmatrix}}_{\text{patch token}} + \underbrace{\begin{bmatrix} 0.10 \\ 0.04 \\ -0.12 \\ \vdots \end{bmatrix}}_{\text{position vector}} = \underbrace{\begin{bmatrix} 0.33 \\ -1.01 \\ 0.75 \\ \vdots \end{bmatrix}}_{\text{position-aware token}}.$$

The resulting tensor has shape $1 \times 145 \times 192$.

2.3 Step 3 — Transformer Block (one of six)

2.3.1 LayerNorm

LayerNorm rescales each token to zero mean and unit variance:

$$\begin{bmatrix} 0.33 \\ -1.01 \\ 0.75 \\ 2.40 \\ \vdots \\ -0.31 \end{bmatrix} \xrightarrow{\text{LayerNorm}} \begin{bmatrix} 0.12 \\ -0.89 \\ 0.54 \\ 1.87 \\ \vdots \\ -0.48 \end{bmatrix}.$$

2.3.2 QKV Projection

A single linear layer $\mathbb{R}^{192} \to \mathbb{R}^{576}$ produces queries, keys and values stacked together (since $192 \times 3 = 576$). The result is reshaped into $h = 3$ heads of width $d_h = 64$ each.

For Head 1, the CLS token's query / key / value vectors look like:

$$\mathbf{q}_{\text{cls}} = \begin{bmatrix} 0.5 \\ -0.2 \\ 0.8 \\ \vdots \end{bmatrix}, \quad \mathbf{k}_1 = \begin{bmatrix} 0.3 \\ 0.1 \\ 0.7 \\ \vdots \end{bmatrix}, \mathbf{k}_2 = \begin{bmatrix} -0.4 \\ 0.6 \\ 0.1 \\ \vdots \end{bmatrix}, \mathbf{k}_3 = \begin{bmatrix} 0.6 \\ -0.3 \\ 0.9 \\ \vdots \end{bmatrix}.$$

2.3.3 Attention scores

The score from the CLS token to patch j is the dot product $\mathbf{q}_{\text{cls}} \cdot \mathbf{k}_j$. For example,

$$\mathbf{q}_{\text{cls}} \cdot \mathbf{k}_1 = (0.5)(0.3) + (-0.2)(0.1) + (0.8)(0.7) + \cdots = 0.15 - 0.02 + 0.56 + \cdots = 2.31,$$

and similarly $\mathbf{q}_{\text{cls}} \cdot \mathbf{k}_2 = 0.45$, $\mathbf{q}_{\text{cls}} \cdot \mathbf{k}_3 = 3.87$.

We scale by $1/\sqrt{d_h} = 1/\sqrt{64} = 0.125$:

$$\big[2.31,\ 0.45,\ 3.87,\ \ldots\big] \times 0.125 = \big[0.289,\ 0.056,\ 0.484,\ \ldots\big].$$

Applying softmax turns these into attention weights that sum to 1:

$$\text{softmax}\big[0.289,\ 0.056,\ 0.484,\ \ldots\big] = \big[0.22,\ 0.05,\ 0.41,\ \ldots\big].$$

So in this example the CLS token pays 41% of its attention to patch 3, 22% to patch 1, and only 5% to patch 2. The full attention tensor over all heads has shape $1 \times 3 \times 145 \times 145$. 3 for the attention heads and 145 by 145 for every tokens attention to every other token.

2.3.4 Weighted sum of values

With value vectors

$$\mathbf{v}_1 = \begin{bmatrix} 0.1 \\ 0.8 \\ -0.3 \\ \vdots \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} 0.5 \\ -0.2 \\ 0.7 \\ \vdots \end{bmatrix}, \mathbf{v}_3 = \begin{bmatrix} -0.4 \\ 0.6 \\ 0.9 \\ \vdots \end{bmatrix},$$

the new CLS representation is

$$\mathbf{h}_{\text{cls}} = \begin{bmatrix} 0.022 \\ 0.176 \\ -0.066 \end{bmatrix} + \begin{bmatrix} 0.025 \\ -0.010 \\ 0.035 \end{bmatrix} + \begin{bmatrix} -0.164 \\ 0.246 \\ 0.369 \end{bmatrix} + \cdots = \begin{bmatrix} -0.117 \\ 0.412 \\ 0.338 \\ \vdots \end{bmatrix}.$$

Every token in the sequence undergoes the same kind of weighted mixing. The output of attention has shape $1 \times 145 \times 192$. The CLS now contains information from patches it attended to and each token is now a weighted mix of other tokens' values.

2.3.5 Residual connection

The block adds the attention output back to its input, so the CLS row becomes

$$\begin{bmatrix} 0.33 \\ -1.01 \\ 0.75 \\ \vdots \end{bmatrix} + \begin{bmatrix} -0.117 \\ 0.412 \\ 0.338 \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.213 \\ -0.598 \\ 1.088 \\ \vdots \end{bmatrix}.$$

2.3.6 MLP

The position-wise MLP expands the channel dimension by 4× and projects back:

$$\mathbb{R}^{192} \xrightarrow{\text{Linear}} \mathbb{R}^{768} \xrightarrow{\text{GELU}} \mathbb{R}^{768} \xrightarrow{\text{Linear}} \mathbb{R}^{192},$$

followed by another residual connection. The output again has shape $1 \times 145 \times 192$.

2.3.7 Stacking the blocks

The block above is repeated $L = 6$ times. Earlier blocks tend to capture low-level structure (edges, textures), while later blocks capture more abstract concepts. After the final block and a closing LayerNorm, suppose the CLS row reads

$$\mathbf{h}_{\text{cls}} = \begin{bmatrix} 0.82 \\ -0.45 \\ 1.23 \\ 0.07 \\ -0.91 \\ \vdots \\ 0.34 \end{bmatrix} \in \mathbb{R}^{192}.$$

2.4 Step 4 — DINO Projection Head

A two-layer MLP takes the 192-dim backbone output to 128 dimensions.

First linear layer ($\mathbb{R}^{192} \to \mathbb{R}^{256}$):

$$\mathbf{h}_{\text{cls}} = \begin{bmatrix} 0.82 \\ -0.45 \\ 1.23 \\ \vdots \end{bmatrix} \longmapsto \begin{bmatrix} 0.55 \\ -0.23 \\ 0.91 \\ 1.04 \\ \vdots \\ -0.67 \end{bmatrix} \in \mathbb{R}^{256}.$$

GELU activation:

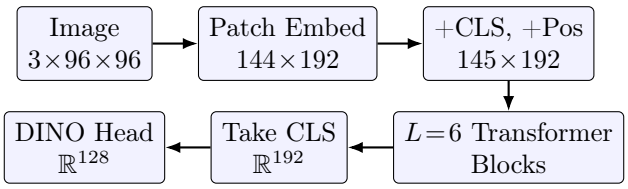
$$\begin{bmatrix} 0.55 \\ -0.23 \\ 0.91 \\ 1.04 \\ \vdots \\ -0.67 \end{bmatrix} \xrightarrow{\text{GELU}} \begin{bmatrix} 0.42 \\ -0.05 \\ 0.78 \\ 0.95 \\ \vdots \\ -0.12 \end{bmatrix}.$$

Second linear layer ($\mathbb{R}^{256} \to \mathbb{R}^{128}$):

$$\mathbf{z} = \begin{bmatrix} 0.73 \\ -0.31 \\ 0.44 \\ 0.12 \\ -0.88 \\ 0.56 \\ \vdots \\ 0.21 \end{bmatrix} \in \mathbb{R}^{128}.$$

This 128-dimensional vector $\mathbf{z}$ *is* the image's representation in the learned embedding space. Two crops of a dog should give similar $\mathbf{z}$'s; a dog and a truck should give very different ones.

2.5 Forward Pass at a Glance



3 Backward Pass

DINO is a self-distillation objective. Two augmented crops of the same image are produced: one is fed to the **student** network, the other to the **teacher**. The student is updated by gradient descent to match the teacher's output distribution; the teacher is updated as a slow exponential moving average (EMA) of the student.

For our running example, suppose both crops show a cat, and the two networks output

$$\mathbf{z}_s = \begin{bmatrix} 0.73 \\ -0.31 \\ 0.44 \\ \vdots \end{bmatrix}, \qquad \mathbf{z}_t = \begin{bmatrix} 0.80 \\ -0.25 \\ 0.50 \\ \vdots \end{bmatrix}, \quad \in \mathbb{R}^{128}.$$

3.1 Step 1 — Compute the DINO Loss

Teacher: center, then sharpen with $\tau_t = 0.04$. A running mean $\mathbf{c}$ is subtracted from the teacher logits to prevent collapse:

$$\mathbf{z}_t - \mathbf{c} = \begin{bmatrix} 0.80 \\ -0.25 \\ 0.50 \\ \vdots \end{bmatrix} - \begin{bmatrix} 0.10 \\ 0.05 \\ -0.02 \\ \vdots \end{bmatrix} = \begin{bmatrix} 0.70 \\ -0.30 \\ 0.52 \\ \vdots \end{bmatrix}.$$

Dividing by $\tau_t = 0.04$ and softmax-ing gives a very peaked target:

$$\mathbf{p}_t = \mathrm{softmax}\big[\,17.5,\ -7.5,\ 13.0,\ \ldots\big] = \big[\,0.45,\ 0.00,\ 0.32,\ \ldots\big].$$

Student: sharpen with $\tau_s = 0.10$.

$$\log \mathbf{p}_s = \log \mathrm{softmax}\big[\,7.3,\ -3.1,\ 4.4,\ \ldots\big] = \big[\,-0.52,\ -10.82,\ -3.42,\ \ldots\big].$$

Cross-entropy loss.

$$\begin{aligned} \mathcal{L} &= -\sum_{k=1}^{128} p_{t,k} \log p_{s,k} \\ &= -\big[\,(0.45)(-0.52) + (0.00)(-10.82) + (0.32)(-3.42) + \cdots\big] \\ &= -\big[\,-0.234 + 0.000 + (-1.094) + \cdots\big] \\ &\approx 1.328. \end{aligned}$$

This is the positive number we want to drive down.

3.2 Step 2 — Gradient at the Output

For a softmax-cross-entropy loss the gradient at the student logits has a particularly clean form. With student probabilities (already computed) $\mathbf{p}_s = [\,0.38,\ 0.01,\ 0.28,\ \ldots\,]$ and teacher target $\mathbf{p}_t = [\,0.45,\ 0.00,\ 0.32,\ \ldots\,]$, we get

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}_s} \propto \mathbf{p}_s - \mathbf{p}_t = \begin{bmatrix} 0.38 \\ 0.01 \\ 0.28 \\ \vdots \end{bmatrix} - \begin{bmatrix} 0.45 \\ 0.00 \\ 0.32 \\ \vdots \end{bmatrix} = \begin{bmatrix} -0.07 \\ +0.01 \\ -0.04 \\ \vdots \end{bmatrix}.$$

Reading this off entry by entry: "the student is putting too little mass on dimension 0 (push it up by 0.07), slightly too much on dimension 1 (pull it down by 0.01), and too little on dimension 2 (push it up by 0.04)."

3.3 Step 3 — Backpropagation Through the Network

The chain rule then walks this gradient *backwards* through every layer of the student. For a generic linear layer $\mathbf{y} = \mathbf{W}\mathbf{x}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{y}} \mathbf{x}^\top, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}^\top \frac{\partial \mathcal{L}}{\partial \mathbf{y}}.$$

For GELU, $\partial \mathcal{L}/\partial \mathbf{x} = (\partial \mathcal{L}/\partial \mathbf{y}) \odot \mathrm{GELU}'(\mathbf{x})$. Residual connections simply pass the upstream gradient through both unchanged *and* into the sub-layer, which is what makes deep networks trainable.

The gradient therefore flows in this order:

DINO head $\to$ LayerNorm $\to$ Block 6 $\to \cdots \to$ Block 1 $\to$ Pos. Embed $\to$ Patch Embed Conv2d.

3.4 Step 4 — Optimizer Step (Student Only)

Pick any single weight w inside the student. Suppose its current value and accumulated gradient are

$$w = 0.342, \qquad \frac{\partial \mathcal{L}}{\partial w} = 0.0015.$$

With learning rate $\eta = 5 \times 10^{-5}$ a vanilla SGD update is

$$w \;\leftarrow\; w - \eta \frac{\partial \mathcal{L}}{\partial w} = 0.342 - (5{\times}10^{-5})(0.0015) = 0.342 - 7.5{\times}10^{-8} \approx 0.341999925.$$

Each individual update is tiny, but with millions of parameters and thousands of steps the network steadily improves. (In practice we use AdamW, which adds momentum and per-parameter adaptive scaling on top of this.)

3.5 Step 5 — EMA Teacher Update (No Gradients)

The teacher is *never* updated by gradient descent. Instead, after each optimizer step we slowly blend the student into it. With EMA momentum $m = 0.996$ and the same example weight,

$$w_{\text{teacher}} \;\leftarrow\; m\, w_{\text{teacher}} + (1-m)\, w_{\text{student}} = (0.996)(0.340) + (0.004)(0.342) = 0.33864 + 0.001368 = 0.340008.$$

The teacher moves only 0.4% of the way toward the student per step, which gives the student a slowly-changing, smoothed target to chase — essential for stable self-distillation.

4 One Training Step, End to End

Putting all of the above together, a single training step looks like:

1. Take one image and produce two random crops, e.g. Crop A and Crop B, each of shape $3{\times}96{\times}96$.
2. **Teacher forward (no grad)**: Crop A $\rightarrow$ $\mathbf{z}_t \in \mathbb{R}^{128}$, e.g. $[\,0.80,\ -0.25,\ 0.50,\ \ldots\,]$.
3. **Student forward**: Crop B $\rightarrow$ $\mathbf{z}_s \in \mathbb{R}^{128}$, e.g. $[\,0.73,\ -0.31,\ 0.44,\ \ldots\,]$.
4. Center and sharpen $\mathbf{z}_t$, sharpen $\mathbf{z}_s$, and compute the cross-entropy loss $\mathcal{L} \approx 1.328$.
5. **Backward**: gradient at the output is $\mathbf{p}_s - \mathbf{p}_t = [\,-0.07,\ +0.01,\ -0.04,\ \ldots\,]$; the chain rule propagates it back through every layer of the student.
6. **Optimizer**: each student weight is nudged by $-\eta\, \partial\mathcal{L}/\partial w$, e.g. $0.342 \rightarrow 0.341999925$.
7. **EMA**: each teacher weight slides 0.4% toward the student, e.g. $0.340 \rightarrow 0.340008$.

Repeated over many images and many steps, this procedure pushes the student to map all crops of the same image (and, by generalization, all images of the same semantic class) to nearby points in $\mathbb{R}^{128}$. The centering and sharpening prevent the trivial collapse $\mathbf{z} \equiv \text{const}$. The result is exactly what we want from a self-supervised feature extractor: a representation in which semantically similar images are close, learned without a single label.